Division One College Basketball Tournament Random Forest Model

Evan Litzer

I. INTRODUCTION

Every March, the most exciting and unpredictable sports event graces our television sets—so magical that people flood to the TruTV channel to embrace its glory. The division 1 college basketball tournament, informally known as "March Madness," fields 68 of the top college basketball teams to compete for a national title in a single elimination, advancing format. Millions of Americans rush to fill out their brackets, dreaming of that perfect bracket while competing in pools for the best prediction. It's a part of the American culture. So why has a perfect bracket never been achieved? Can an advanced model trained with previous tournament data lead to higher performing brackets? And what statistics can help differentiate between a loser and winner in a matchup in a particular round? I set out to answer these questions through this project and delved into the most unpredictable sports rabbit-hole to ever exist.

II. DATA

A. Finding a Dataset

My data originated from the March Machine Learning Mania 2025 competition on the 'Kaggle' dataset website. The website holds this annual feature prediction competition--challenging its users to develop a machine learning model that can predict the best bracket probabilities for tournament matchups. I used their available dataset but would structure my model a bit differently. The dataset was constructed in the csv (comma separated value) file format.

B. Team Statistics and Tournament Structure

The data is construed through the 21 years of regular and tournament season data between 2003 and 2024 inclusive but excluding 2020 due to the tournament being cancelled. I recognize and distinguish between teams through a team ID identification system. Then I track team statistics in the regular season through average game accumulation. These stats include average points scored and allowed, along with shooting efficiency through field-goal percentage, three-point percentage, and free-throw percentage. There also were volume shooting stats depicted through normal and three-point field-goals made and attempted, along with free throws. Finally, there were possession stats demonstrated by turnovers, assists, steals, fouls, and offensive/defensive rebounding.

The model trains not on the raw numbers of the regular season game averages but rather computed them as differences to infer what statistics equated to winning. The tournament structure for each year is defined by the tournament slots, which allow my model to build their own forecasted brackets on the fly. Yearly tournament data also exists for speculation comparison and training validation. This includes both round matchups and seeding information. Ranking data was also offered, though I used it minimally, lightly incorporating it into my ELO System.

C. Data Manipulation

In order for the model to treat each game fairly and consider every perspective, data-engineering must occur. This first method includes overtime conversion, which converts any game's stats that had any amount of overtime to its forty-minute mean. Then, each game's instance in my dataset had to be doubled. This allowed the model to intake both the loser's and winner's turn at being Team A vs Team B, and also gave the model double the amount of figures to deliberate on.

As aforementioned, the model observes analytical differences instead of raw figures. This also encapsulates seed difference. Each of the

samples were labeled with a binary outcome: 1 if the team perspective won and 0 if they lost. These labels are flipped for each game's mirrored counterpart.

D. ELO Rating System

The ELO rating system was used to evaluate each team's relative strength over the course of the entire season based on game outcomes and associated margin of victories. All team ELOs are initialized to 1000 to begin the season, besides the AP top 25 teams, who received a variable boost based on rank.

The ratings are updated after each regular season game. The marginal update amount considered the labeled outcome, margin of victory, and home court advantage. For margin of victory, larger wins caused bigger shifts compared to squeaky wins. Home court advantage applied a temporary 100 point boost to the home team's ELO rating.

The probability that Team A defeats Team B is equated through the logistic function:

$$\mathbf{P(A)} = 1/1 + 10^{((E_B - E_A + HCA)/400)}$$

Where E_B and E_A are the ELO ratings of team A and team B respectively. HCA is home court adjustment, set to 100 and its sign switched based on if team A is at home. If team A has a much higher ELO, then P(A) will be closer to 1. If team A is weaker or playing away, then it will be lower. This expected probability becomes the baseline against which the actual outcome is compared.

The ELO update equation affects how much each team's ELO changes after the game, scaled by margin of victory.

$$\textbf{ELO Change} = K * (1-P(A)) * \log(\text{margin} + 1) * (2.2/(0.001 * \text{ELO\_Diff} + 2.2))$$

K is the learning rate defining how much ELO should change. Margin is the point differential of the game and ELO difference is self-named. P(A) is the probability team A beats team B, as shown above. This update mechanism ensures that ELO ratings do not linearly reflect the outcome of a singular game, but also how convincing the win is and how unexpected it was based on previous rating.

Over the course of the season, these updates will accurately reflect the competitive strength of every team. The final values, computed as EloDiff (ELO difference), is an essential data feature in my model. In fact, I found it to be the most effective statistic.

III. METHODS

My goal for this project was to predict the NCAA tournament, and bundle those predictions into a fully simulated bracket. The model was evaluated over the 21 years in the dataset, though different years were used for different purposes: training and testing. The evaluation of its performance reflects common bracket-creating platforms.

A. Feature Construction

For every tournament matchup, a feature vector was generated composed of all the regular season statistical data that was composed for the year. This includes game-stat averages, as well as seed and ELO differences. This set the environment and groundwork for the model, giving it the data toolset it needed to begin making data-backed predictions.

B. Model Training

To train the predictive model, I proposed the matchup prediction as a binary decision. Each instance the model received represented a matchup between two teams, and the label (either 1 or 0) indicated whether the first team won the game to the instance order.

Historical tournament matchups spanning the 2003-2015 time period were utilized as the training set. These matchup instances included all the regular season data features for both teams. The matchups were mirrored—each game had two instances for two different perspectives.

My training method involved Random Forest classifiers. Random Forest models build countless decision trees on random subsets of the data and features of my dataset, then condense their predictions and take a majority vote. They are effective for my project envisionment because it

can weight features differently and learn nonlinearly. For example, if the model observes teams that turn the ball over more and have a low game-score average lose more often, then it may shift away from choosing the team in testing sets.

So, in short, the Random Forest model used the generated feature vectors for both teams to train by building decision trees. The labels representing actual tournament games then validate the training process. Then the model applies this gained expertise to the testing sets.

C. Bracket Simulation and Evaluation

Once the model was trained, I then applied it to simulate the tournament brackets for each season from 2016 to 2024, excluding 2020 due to the coronavirus. Each year's bracket was simulated using the model's binary decisions to determine the victor of each matchup. This was a carry-over process, in particular, a team that the model chooses to win the matchup advances to the next round.

This process is built upon the real-life seeding of the teams and the tournament slots file the model will eventually use for validation. The function simulate_bracket() handles this intricate and essential portion of my model. It first resolves the play-in games, which I skip due to the layout of my tournament files. Then it advances the winner of each matchup determined by the model to the next tournament slot representing the sequential round.

To evaluate the model's performance on a bracket for a certain year, I used a weighted scoring system that follows the scoring rules below.

(R1) Round of 32: 1 Point

(R2) Sweet 16: 2 Points

(R3) Elite 8: 4 Points

(R4) Final 4: 8 Points

(R5) Championship 2: 16 Points

(R6) Champion: 32 Points

I designed it like this to match the system of popular bracket-creation applications. This way, I can compare my model's performance to the millions of humans who fill out their brackets yearly. The average bracket score across the 9 test years served as the indicator for model performance and was used to select the best model during randomized training trials.

To better optimize model performance, I administered a randomized hyperparameter search across 50 trials. I ran this search hundreds of time to find the model with the best parameter weights. When the final model was selected and validated through realistic tournament simulations, I then moved on to analyzing its results through its parameters and success among the testing brackets.

IV. RESULTS

My model was graded on its average bracket validation score across the six testing years. This did cause some outliers to exist when measuring averages, but I preferred this over grading by individual year.

My best performing model averaged around 48 points per bracket. The results varied significantly by year, as observed in the graph depicted in figure 1. In 2016, the model scored a high 87 points, but around 32 points in 3 different years, and around 40 in 2 more. This shows that the model had very volatile scoring over different brackets.

The second highest score was composed in 2019, putting up around 65 points. Bracket scoring was overly dependent on later rounds, with the champion slot being equivalent to matching every round of 32 slot. In both high-scoring years, at least one slot in the championship was allotted correctly.

I also calibrated random forest models for each round of the tournament for compartmentalization. My goal was to isolate variables and find which data features were important in which round. This way, I can observe patterns that accumulate by round, applying it past the generalized tournament.

The results from this pigeonholed model are gathered in the Figure 2 heatmap. ELO difference, average point margin difference, and seed difference comprise the three most prominent tournament data features. But as the tournament

progresses, they progressively get less meaningful. This is due to the lower seeds making the later rounds, but it's still crucial to note how other team statistics replace them. Rebounding, three point percentage, and assists contain their highest weight in the championship round. Field-goal percentage, steals, and turnovers do not have much weight in any rounds. Notice that despite some aspects of a team stats are weighted significantly more and less, yet they all play a role in the model determining a winner from a loser. These decimals are not positive nor negative, only how much a model leans on them to formulate a decision.

When applying my model to the 2025 bracket (Figure 3), it scores a testing-set smashing 101 points. The reason why I could not include the bracket in the testing set was due to the tournament not occurring before the dataset was made, as the Kaggle competition's purpose was to predict this year's bracket. I manually scored the model's bracket output, and it was my highest by 15 points. This could be due to the chalkiness of the year, yet the model did choose some upsets.

The 2025 model prediction obliterated the first round, getting an astonishing 25 of 32 matchups correct. It then begins to falter in the sweet 16 and elite 8, but still gets half of the Final Four correct. The model ended up picking Houston and eliminating Florida in the Elite 8.

V. DISCUSSION

According to NCAA.com, the average user score for filling out a bracket is around 67 points. If one flipped a coin for each bracket matchup, then they would have an expected score of 31.5. Therefore, my model falls in about the middle between random and average user-score according to my testing set mean.

The model was much evidently better in earlier round matchups than later round, which is evidenced by the 2025 bracket. Overfitting may have occurred with my validation system, which will require reworking.

I believe that testing my model with humanistic scores worked great for comparing it to human performance, but in the end, hurt the model during training. Weighing the later rounds more than the earlier rounds introduced compounding error that loomed on the validation scores of the bracket. The model made decisions on which teams to advance, yet both teams usually have a fairly similar chance to advance after the first round. The Kaggle competition accepted probabilities for submission, but I wanted to take it a step further and encourage my model to fill out its own brackets.

The low scores should not discourage the approval of my model. It was still able to predict bracket games at a 72% percent rate. Picking the higher seed would yield around 70% success rate. This doesn't bode well for showing my model was impactful, but I am satisfied knowing it chose upsets and took verbose risk. My model also presented which data points were more useful for each round.

With more data and implementation, my model could improve vastly. I am planning on expanding the model over the summer for next year's tournament. My next ideas are to add player stats and conference tournaments to the dataset mix. I also envision isolating Cinderella teams to pinpoint a trend I can then identify and apply to future tournaments.

The March Madness tournament was built to be unpredictable, and this project expressed that. You could combine the memory of all computers on Earth with every available datapoint, and it still wouldn't be close to perfection. This is due to the nature of basketball and how there are some things that cannot be modeled. Some examples include injuries, schemes, game plans, player styles, referee variance, and team performance variance. All models are wrong, but some are useful. Flawlessness may never be achieved, but random forest models can help inch brackets closer to correctness and discover evident patterns.

These patterns were outlined in Figure 2, but show how some datapoints are more marginally impactful than others when determining an advancing victor.

In a tournament defined by unprecedented unpredictability, even the slightest edge offers

significant value. My model is not about magically predicting the future but understanding it more deeply. With the growing implementation of further features, and a quick reworking of the scoring system, it can become not only more accurate, but more insightful. Then maybe one day, I can win my family's bracket pool.

FIGURES

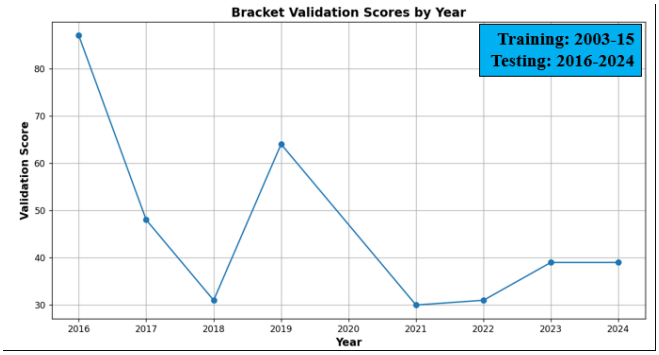

Figure 1: Model Performance Validation Scores through 2016-2024

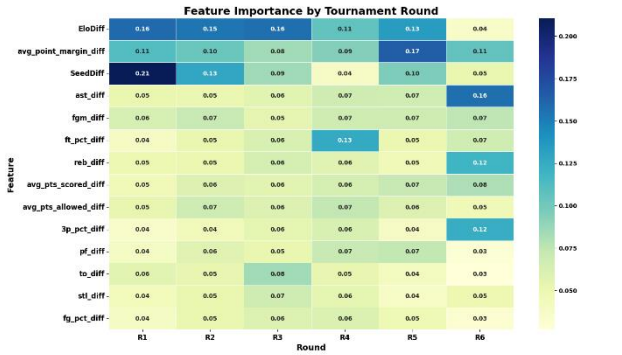

Figure 2: Round by Round Random Forest Variables

Figure 3: 2025 NCAA Tournament Model Prediction

REFERENCES

Wilco, Daniel. "Here's How Your March Madness Bracket Will Do If You Only Pick the Better-Seeded Team." NCAA.Com, NCAA.com, 14 Mar. 2021, www.ncaa.com/news/basketball-men/bracketiq/2021-02-12/heres-how-your-march-madness-bracket-will-do-if-you-only-pick-better-seeded.